MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING

UNIT 1: INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Comprehensive Examination Notes, Formal Definitions, Agent Architectures & Solved Question Bank

AUDITED AGAINST COURSE POLICY (2026-27), AIMA (RUSSELL & NORVIG 3RD ED) & UNIVERSITY EXAM PAPERS

EXAMINATION MANUAL PREFACE & SCOPE AUDIT:
This study manual provides exhaustive coverage of **Unit 1: Introduction to Artificial Intelligence** as prescribed by the official university course policy (SVKM's NMIMS MPSTME & Mumbai University). It is rigorously aligned with *Artificial Intelligence: A Modern Approach (AIMA 3rd Edition)* by Stuart Russell & Peter Norvig. All Unit 2 search algorithms (BFS, DFS, A*, Minimax) are strictly excluded to maintain pure Unit 1 focus.

TABLE OF CONTENTS

Section / Topic Description	Page
1.0 Foundations, Definitions & Historical Evolution of AI	Page 2
2.0 AI Pillars, Modeling, Inference, Learning & AI-ML-DL Hierarchy	Page 3
3.0 Intelligent Agents & The Concept of Rationality	Page 4
4.0 PEAS Framework & Task Environment Taxonomy	Page 5
5.0 Structure & Architectures of Intelligent Agents (5 Agent Types)	Page 7
6.0 Vacuum-Cleaner World Case Study & 8-State Space Transitions	Page 10
7.0 Examination Preparation: Solved PYQ Bank & Syllabus Checklist	Page 11

1.0 FOUNDATIONS, DEFINITIONS & HISTORICAL EVOLUTION OF AI

Artificial Intelligence (AI) is a universal field of computer science concerned with designing intelligent agents—computational entities that perceive their environment and execute actions that maximize their probability of successfully achieving their goals. The term was officially coined by **John McCarthy in 1956** at the landmark Dartmouth Summer Research Project.

Canonical Definitions from AI Pioneers:

> • **John McCarthy (1956):** 'The science and engineering of making intelligent machines, especially intelligent computer programs.'
> • **Ray Kurzweil (1990):** 'The art of creating machines that perform functions that require intelligence when performed by people.'
> • **Elaine Rich & Kevin Knight (1991):** 'The study of how to make computers do things at which, at the moment, people are better.'
> • **Richard Bellman (1978):** 'The automation of activities that we associate with human thinking, activities such as decision-making, problem solving, learning...'
> • **Patrick Winston (1992):** 'The study of the computations that make it possible to perceive, reason, and act.'
> • **Nils Nilsson (1998):** 'AI is concerned with intelligent behavior in artifacts.'
> • **David Poole et al. (1998):** 'Computational Intelligence is the study of the design of intelligent agents.'

The Four Categorizations / Approaches to AI:

As established by Russell & Norvig (*AIMA*), historical definitions of AI fall into a 2×2 matrix along two core dimensions: (1) **Thought processes vs. Behavior**, and (2) **Human fidelity vs. Ideal Rationality**.

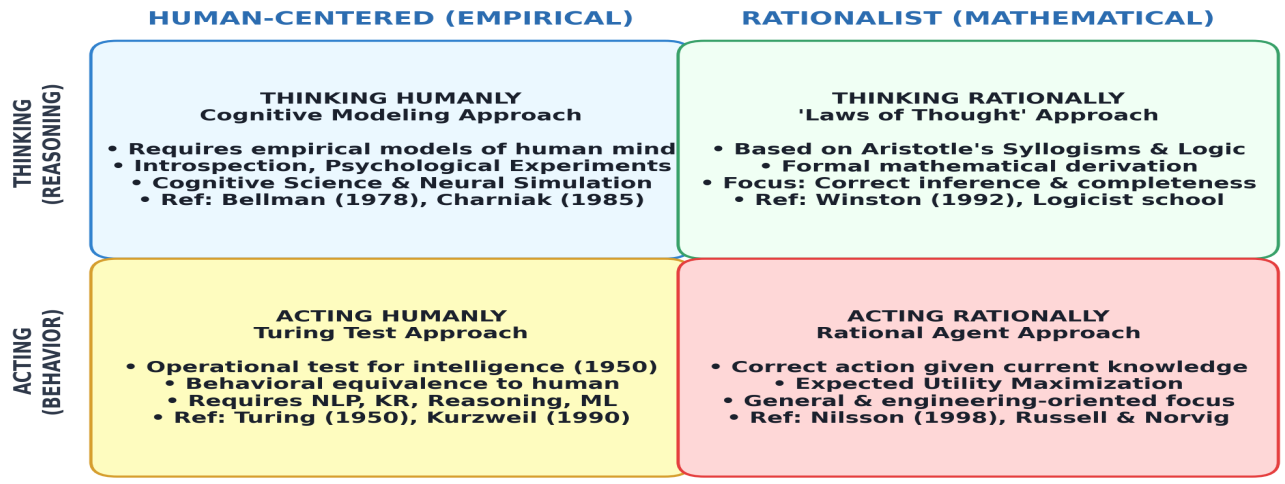


Figure 1.1: The 2×2 Matrix Categorization of AI Approaches (Russell & Norvig).

Detailed Breakdown of the 4 Approaches:

Approach	Focus & Mechanism	Evaluation Basis	Pioneers
Approach Dimension	Core Focus & Scientific Basis	Evaluation Criterion	Key Pioneers / Ref
Thinking Humanly	Cognitive Science: Modeling internal human thought processes via introspection, psychological testing, and neuro-imaging.	Fidelity to human brain reasoning and cognitive timing.	Bellman (1978), Charniak & McDermott (1985)

Approach	Focus & Mechanism	Evaluation Basis	Pioneers
Thinking Rationally	Laws of Thought: Formalizing irrefutable logical deductions using syllogisms and predicate calculus.	Logical correctness and proof completeness.	Aristotle, Winston (1992)
Acting Humanly	Turing Test Approach: Achieving operational behavioral equivalence to a human in communication and problem solving.	Human interrogator cannot distinguish computer from human.	Alan Turing (1950), Kurzweil (1990)
Acting Rationally	Rational Agent Approach: Autonomous action selection to maximize expected utility given current knowledge.	Expected utility maximization in real-world environments.	Nilsson (1998), Poole (1998), Russell & Norvig

The Turing Test & Total Turing Test:

Proposed by **Alan Turing (1950)** in his seminal paper *'Computing Machinery and Intelligence'*, the Turing Test provides a satisfactory operational definition of intelligence, sidestepping philosophical vagueness.

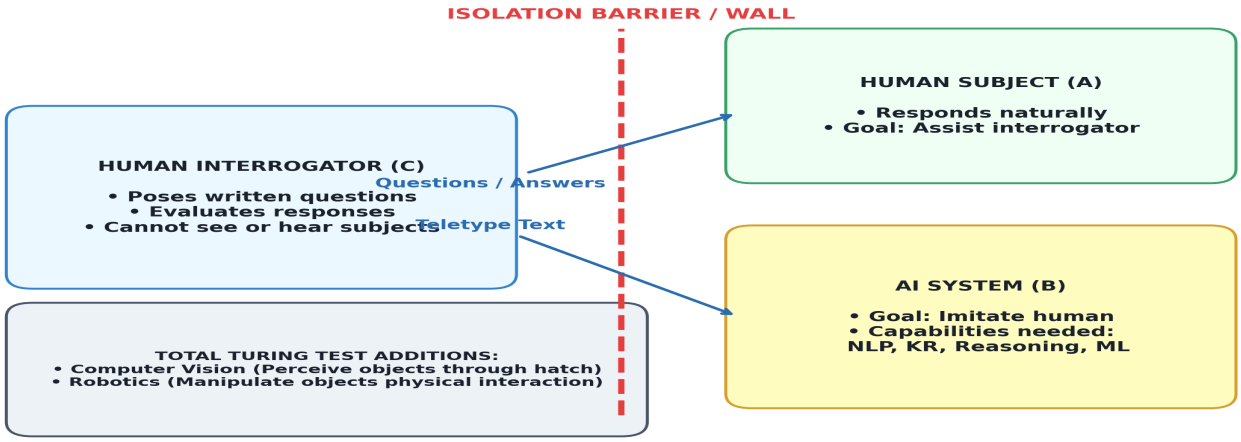


Figure 1.2: The Turing Test and Total Turing Test Operational Environment.

Required AI Capabilities for Turing Tests:

Standard Test (4 Disciplines)	Total Turing Test (2 Additional Disciplines)
Standard Turing Test Capabilities (Text Terminal)	Total Turing Test Capabilities (Physical Hatch / Vision)
1. **Natural Language Processing (NLP):** To communicate fluently in human language. 2. **Knowledge Representation:** To store structured information prior to and during the test. 3. **Automated Reasoning:** To use stored knowledge to answer questions and draw logical conclusions. 4. **Machine Learning:** To adapt to new circumstances, detect patterns, and learn from experience.	5. **Computer Vision:** To perceive physical objects, visual signals, and gestures passed through the hatch. 6. **Robotics:** To manipulate physical objects, navigate space, and demonstrate motor control.

2.0 AI PILLARS, MODELING, INFERENCE, LEARNING & SUBSET RELATIONSHIPS

The Three Core Pillars of Artificial Intelligence:

1. **Modeling:** The formal mathematical or symbolic representation of real-world domain knowledge, world states, constraints, and relationships.
2. **Inference:** The computational deduction or search engine that operates upon internal models to derive new facts, answer

queries, or plan action sequences.
3. **Learning:** The adaptive process through which an agent updates its internal models and inference parameters based on environmental feedback and experience.

Nested Relationship: AI vs. Machine Learning vs. Deep Learning:

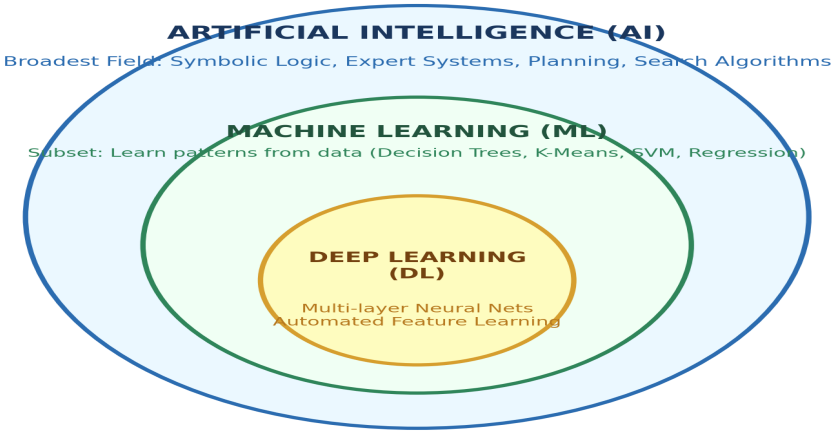


Figure 2.1: Concentric Container View of AI, Machine Learning, and Deep Learning.

Dimension	Artificial Intelligence	Machine Learning	Deep Learning
Feature / Dimension	Artificial Intelligence (AI)	Machine Learning (ML)	Deep Learning (DL)
Scope & Definition	Overarching field building intelligent systems that perceive, reason, and act.	Statistical subset of AI that learns patterns from data without explicit coding.	Specialized subset of ML using deep neural networks with multi-layer feature hierarchies.
Feature Engineering	Manual knowledge engineering & expert rule formulation.	Requires manual extraction of domain-specific features by data scientists.	Automatic representation learning; extracts raw low-to-high level features directly.
Data Dependency	Works on small rule sets & logical axioms.	Requires thousands of structured tabular samples.	Requires massive datasets (millions of unstructured images, audio, text).
Compute Requirements	Standard CPU execution.	Standard multi-core CPUs.	High-performance GPUs / TPUs for heavy matrix operations.

Supplementary Context: Hard Computing vs. Soft Computing:

Parameter	Hard Computing	Soft Computing
Parameter	Hard Computing (Traditional AI)	Soft Computing (Computational Intelligence)
Premise & Logic	Requires precise, crisp, binary logic (True/False; 1/0).	Tolerates imprecision, uncertainty, partial truth, and approximations.
Execution Model	Deterministic, sequential mathematical algorithms.	Parallel, adaptive, bio-inspired stochastic algorithms.
Core Techniques	First-Order Logic, Expert Systems, Symbolic Search.	Artificial Neural Networks (ANN), Fuzzy Logic, Genetic Algorithms (GA).
Noise Tolerance	Low; sensitive to missing or corrupted input data.	High; highly robust and fault-tolerant to noise and ambiguity.

3.0 INTELLIGENT AGENTS & THE CONCEPT OF RATIONALITY

An **Agent** is anything that can be viewed as perceiving its environment through **Sensors** and acting upon that environment through **Actuators**.

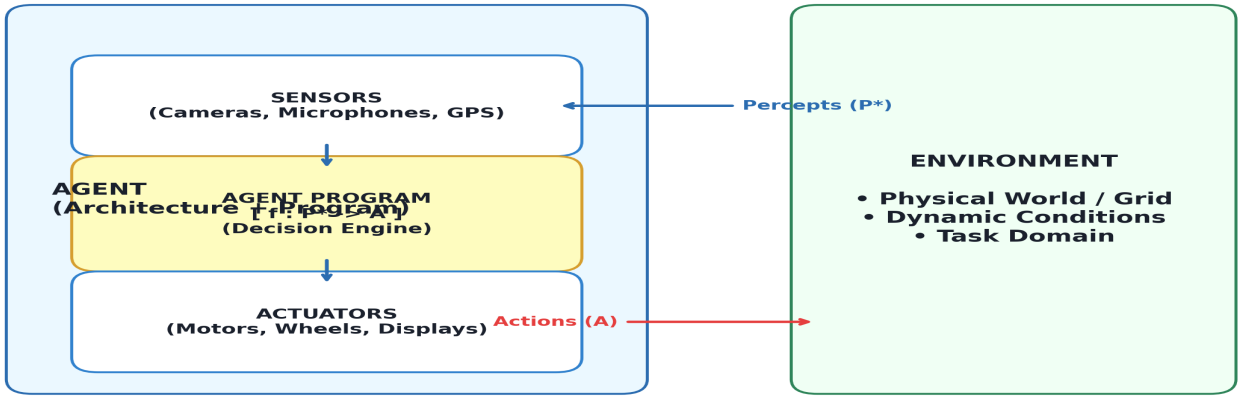


Figure 3.1: The Agent-Environment Interaction Cycle.

Mathematical Formalisms & Architectural Equation:

- **Percept Sequence (P*):** The complete history of all percepts received by the agent from initialization to time *t*.
- **Agent Function (f):** An abstract mathematical description mapping percept histories to actions: $\mathbf{f : P^* \rightarrow A}$
- **Agent Program:** A concrete algorithmic implementation running on a physical computing architecture that realizes *f*.
- **Core Architecture Equation:**

> **AGENT = ARCHITECTURE + AGENT PROGRAM**
> - **Architecture:** The physical computing hardware platform, sensors, actuators, and OS interface.
> - **Agent Program:** The concrete algorithm that processes percepts and executes the agent function *f*.

Concept of Rationality & The 4 Governing Factors:

A **Rational Agent** is one that selects an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence to date and whatever built-in prior knowledge the agent possesses.

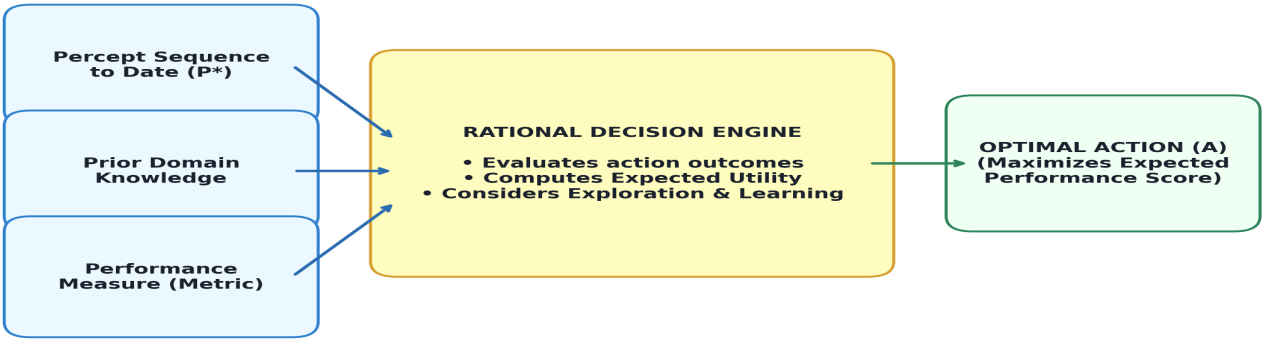


Figure 3.2: Rational Agent Decision Flow and Governing Inputs.

Crucial Exam Distinctions: Rationality vs. Ideal Concepts:

Concept Pair	Theoretical Distinction & Real-World Implication
Concept Pair	Core Distinction & Theoretical Reality
Rationality vs. Omniscience	**Omniscience** knows the *actual outcome* of its actions in advance (requires infinite sensors and zero noise). **Rationality** maximizes *expected performance* based on current evidence. An agent can be perfectly rational without being omniscient.
Rationality vs. Perfection	**Perfection** guarantees flawless outcome success. **Rationality** evaluates decisions based on available evidence, not hindsight failure caused by unobservable stochastic events.
Information Gathering & Exploration	A rational agent must execute **information-gathering actions** (e.g., looking both ways before crossing) and **exploration** to discover unknown environments.
Autonomy	A rational agent possesses **Autonomy** if its behavior is determined by its own learning experience rather than solely by built-in prior knowledge provided by the designer.

4.0 PEAS FRAMEWORK & TASK ENVIRONMENT TAXONOMY

In designing a rational agent, the first step is always to specify the **Task Environment** as fully as possible using the **PEAS Framework** (Performance measure, Environment, Actuators, Sensors).



Figure 4.1: The PEAS Framework Structural Decomposition.

Expanded PEAS Specifications for 6 Benchmark Domains:

Agent Domain	Performance (P)	Environment (E)	Actuators (A)	Sensors (S)
Agent System	Performance Measure (P)	Environment (E)	Actuators (A)	Sensors (S)
Automated Taxi Driver	Safety, destination speed, legal compliance, passenger comfort, max profits.	Roads, city traffic, pedestrians, weather, potholes, police.	Steering wheel, accelerator, brake, signal indicators, horn, display.	Cameras, radar, LIDAR, GPS, speedometer, accelerometer, engine sensors.
Medical Diagnosis System	Patient health recovery, min treatment cost, min false diagnosis penalties.	Hospital, patient symptoms, lab test results, disease database.	Screen display of diagnosis, prescription printing, alert signals.	Keyboard input of symptoms, lab device readings, touchscreen.
Automated Vacuum Cleaner	Cleanliness score, min power consumption, min noise, min time taken.	Room floors (Squares A & B), dirt, obstacles, furniture.	Wheels/Motors, suction pump, brush drives, warning light.	Dirt detection sensor, bump sensor, location cliff sensor, infrared.
Interactive English Tutor	Student test score gain, engagement level, min learning time.	Student user, exercise database, curriculum standards.	Screen display of questions, audio voice synthesis, score feedback.	Keyboard, mouse, microphone for voice input, touchscreen.

Agent Domain	Performance (P)	Environment (E)	Actuators (A)	Sensors (S)
Robotic Part Picker	Percentage of parts correctly binned, placement speed, zero damage.	Conveyor belt with mixed parts, bins, factory floor.	Jointed robotic arm, vacuum gripper, conveyor belt speed control.	Visual camera, conveyor speed encoder, tactile touch sensors.
Internet Shopping Agent	Max cost savings, user item preference match, fast delivery.	E-commerce websites, vendor APIs, pricing databases.	Display recommendations, automated cart addition, API query execution.	HTTP web scraping, JSON API responses, database queries.

Task Environment Dimensions & Taxonomy Tree:

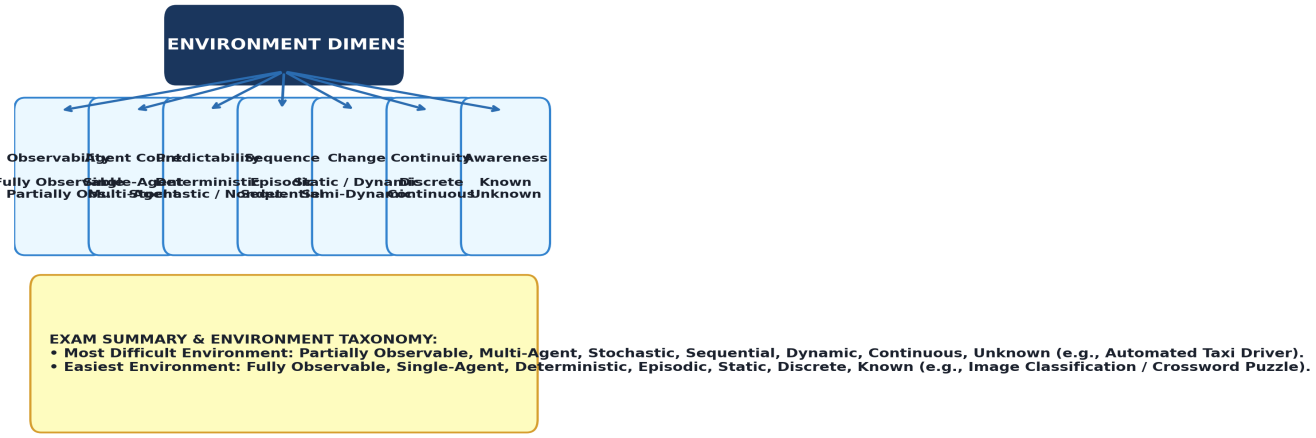


Figure 4.2: Task Environment Dimensions and Taxonomy Summary.

Detailed Definitions of the 7 Task Environment Dimensions:

Dimension	Definition & Distinction	Real-World Examples
Dimension	Definition & Paired Distinction	Concrete Examples
1. Observability	**Fully Observable:** Sensors access complete state of world at any moment. **Partially Observable:** Noise, inaccurate sensors, or unobserved hidden variables.	Fully: Chess board. Partially: Taxi driving, Poker.
2. Agent Count	**Single-Agent:** Agent operates alone without other decision makers. **Multi-Agent:** Multiple agents interacting competitively or cooperatively.	Single: Crossword puzzle. Multi: Chess (competitive), Taxi.
3. Predictability	**Deterministic:** Next state is completely determined by current state & action. **Stochastic / Nondeterministic:** Uncertainty, noise, or random environment changes.	Deterministic: Chess, Vacuum. Stochastic: Taxi driving, Weather.
4. Sequence	**Episodic:** Experience divided into independent atomic episodes; current decision does not affect future episodes. **Sequential:** Current decision affects all future decisions.	Episodic: Defective part picking. Sequential: Chess, Taxi driving.
5. Change Rate	**Static:** Environment does not change while agent is deliberating. **Dynamic:** Environment continuously changes during deliberation. **Semi-Dynamic:** Environment doesn't change, but time score decreases.	Static: Crossword puzzle. Dynamic: Taxi driving. Semi-Dynamic: Timed Chess.
6. Continuity	**Discrete:** Finite, distinct number of states, time steps, and actions. **Continuous:** Continuous range of real-valued state variables and actions.	Discrete: Chess, Crossword. Continuous: Taxi driving, Refinery.
7. Awareness	**Known:** Physics/rules of environment are completely known to designer. **Unknown:** Agent must explore to discover environment laws.	Known: Solitaire card game. Unknown: New video game level.

Verified Task Environment Classification Matrix:

Domain	Observ.	Agent	Predict.	Seq.	Change	Contin.	Aware.
Task Environment	Observable	Agents	Predictability	Sequence	Change Rate	Continuity	Awareness
Chess (with clock)	Fully	Multi (Competitive)	Deterministic	Sequential	Semi-Dynamic	Discrete	Known
Crossword Puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete	Known
Automated Taxi Driver	Partially	Multi (Cooperative/Comp)	Stochastic	Sequential	Dynamic	Continuous	Known
Medical Diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous	Known
Vacuum Cleaner	Fully/Partial	Single	Deterministic	Sequential	Static/Dynamic	Discrete	Known
Part-Picking Robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous	Known
Interactive Tutor	Partially	Multi (Cooperative)	Stochastic	Sequential	Dynamic	Discrete	Known
Internet Shopping	Partially	Multi (Competitive)	Stochastic	Sequential	Dynamic	Discrete	Known

5.0 STRUCTURE & ARCHITECTURES OF INTELLIGENT AGENTS

Agent programs are structured into five basic architectural types listed in order of increasing generality and decision-making capability:

5.1 Simple Reflex Agent:

A **Simple Reflex Agent** selects actions based strictly on the *current percept*, ignoring the rest of the percept history. It relies on condition-action rules (*if condition then action*).

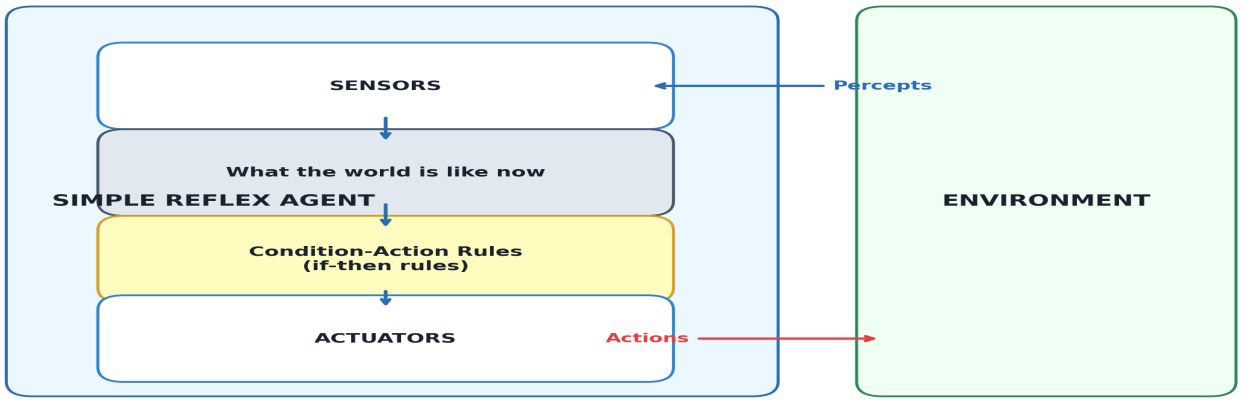


Figure 5.1: Simple Reflex Agent Architecture.

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition-action rules
  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

• **Limitations & Loop Breaking:** Fails in partially observable environments and suffers from unrecoverable infinite loops. However, infinite loops can be broken by introducing **randomized / stochastic action selection**.

5.2 Model-Based Reflex Agent:

A **Model-Based Reflex Agent** maintains an internal state to track unobserved aspects of the current world. It uses two knowledge models: (1) **Transition Model** ('How the world evolves'), and (2) **Sensor Model** ('What my sensors mean').

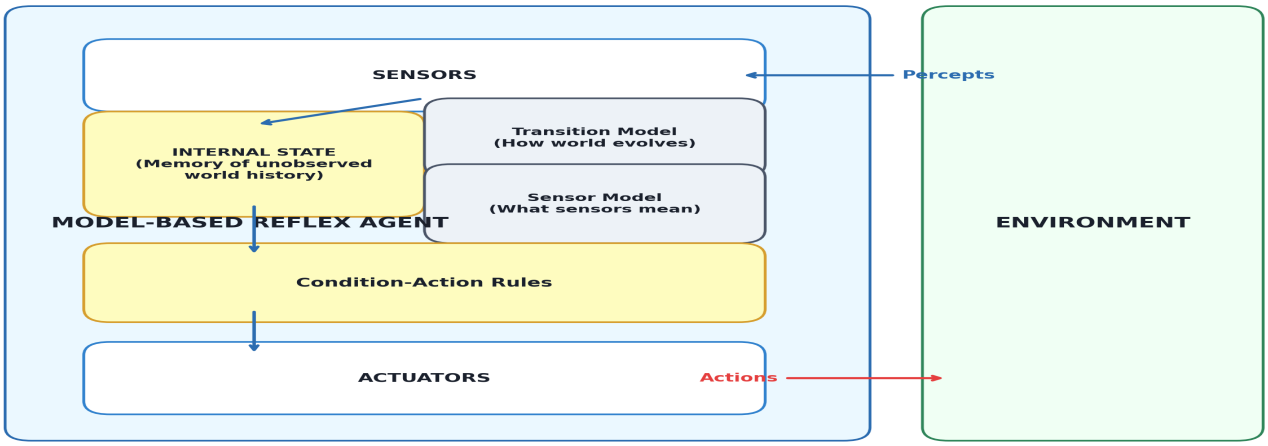


Figure 5.2: Model-Based Reflex Agent Architecture with Internal State.

function MODEL-BASED-REFLEX-AGENT(percept) **returns** an action
persistent: *state* (current world model), *transition_model*, *sensor_model*, *rules*, *action*
state ← UPDATE-STATE(state, action, percept, transition_model, sensor_model)
rule ← RULE-MATCH(state, rules)
action ← rule.ACTION
return action

5.3 Goal-Based Agent:

A **Goal-Based Agent** combines internal state tracking with explicit **Goal Information** describing desirable target situations. It uses search and planning algorithms to evaluate 'What will it be like if I do action A?'

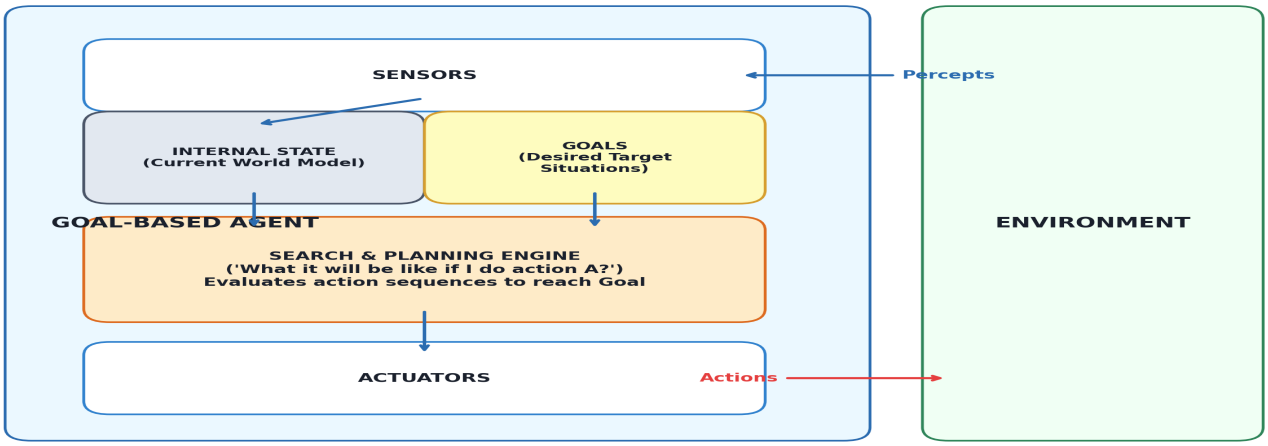


Figure 5.3: Goal-Based Agent Architecture.

5.4 Utility-Based Agent:

A **Utility-Based Agent** uses an internal **Utility Function [U : S → R]** to measure 'how happy' the agent will be in a given state. Enables rational decision-making when multiple goals conflict or require trade-off optimization under uncertainty.

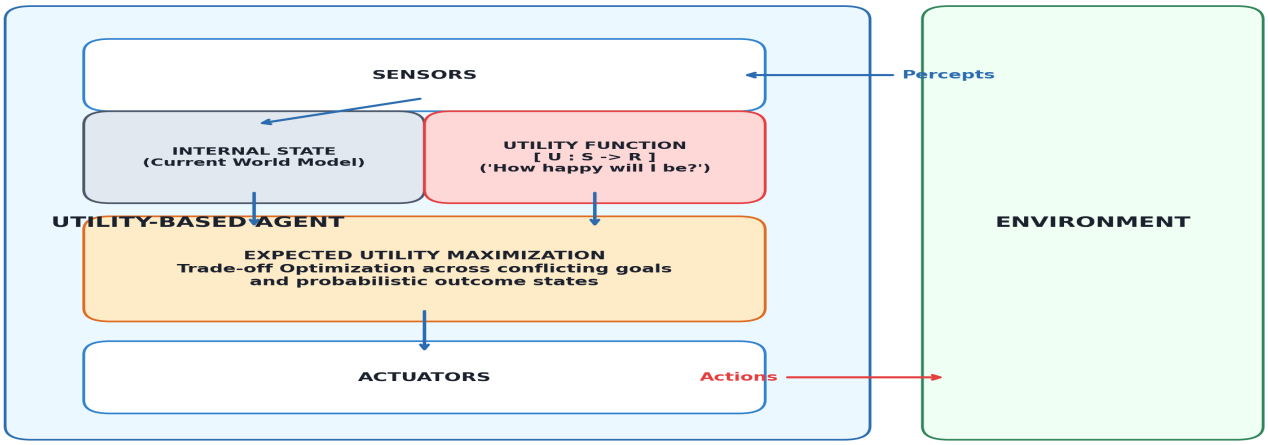


Figure 5.4: Utility-Based Agent Architecture.

5.5 Learning Agent:

A **Learning Agent** is divided into four distinct conceptual components: (1) **Critic**, (2) **Learning Element**, (3) **Problem Generator**, and (4) **Performance Element**.

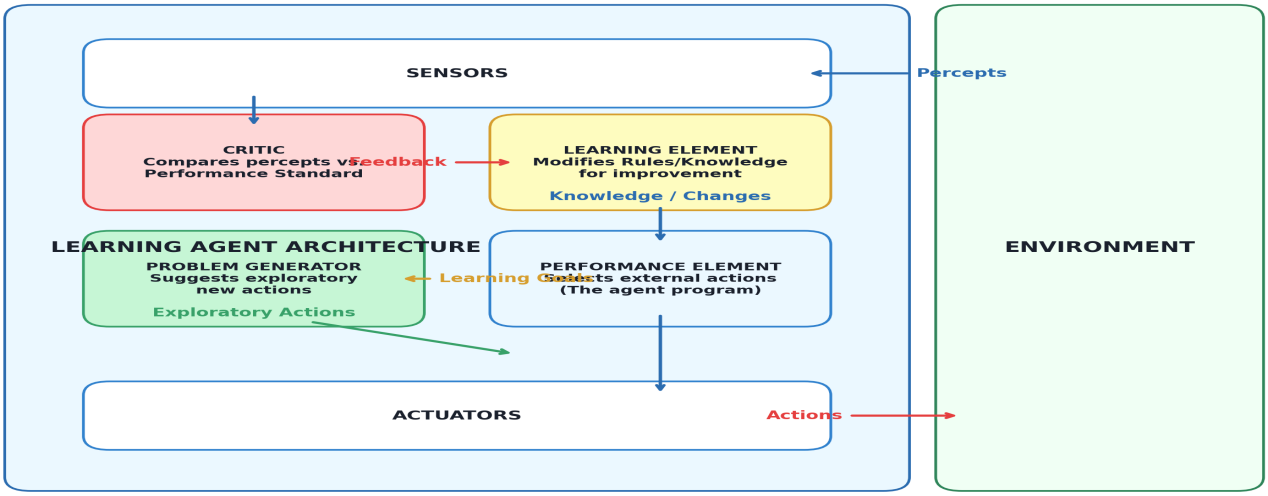


Figure 5.5: Complete Learning Agent Architecture.

Component	Role Description
Component Name	Role & Mechanism within Learning Architecture
Critic	Evaluates agent's behavior against an external **Performance Standard** and provides feedback score.
Learning Element	Processes feedback from Critic and modifies internal knowledge/rules to improve future decisions.
Problem Generator	Suggests exploratory, non-suboptimal actions to discover new experiences and better strategies.
Performance Element	The operational agent program that receives percepts and selects external actions.

Master 5-Way Comparative Matrix of Agent Architectures:

Agent Type	Memory	Decision Basis	Suitability	Flexibility
Agent Architecture	Internal Memory / State	Decision Criterion	Environment Suitability	Flexibility / Adaptability
Simple Reflex	None (Current percept only)	Condition-Action Rules	Fully Observable only	Low (Rigid lookup)
Model-Based Reflex	Maintains World State History	State + Condition Rules	Partially Observable	Moderate (State tracking)
Goal-Based	Maintains World State	Goal Satisfaction (Search/Plan)	Complex / Sequential	High (Goal re-planning)
Utility-Based	Maintains World State	Expected Utility Optimization	Complex / Conflicting Goals	Very High (Trade-off utility)
Learning Agent	State + Performance Standard	Feedback + Learning Element	Unknown / Changing World	Maximum (Self-improving)

6.0 VACUUM-CLEANER WORLD CASE STUDY & 8-STATE SPACE TRANSITIONS

The Vacuum-Cleaner World Benchmark:

- **Environment Geography:** 2 location squares: **A** and **B**.
- **Percept Space:** Tuple *[Location, DirtStatus]* e.g., *[A, Dirty]*.
- **Action Set:** *{Left, Right, Suck, NoOp}*.
- **State Space Calculation:** 2 locations × 2^2 dirt configurations = **8 possible states**.

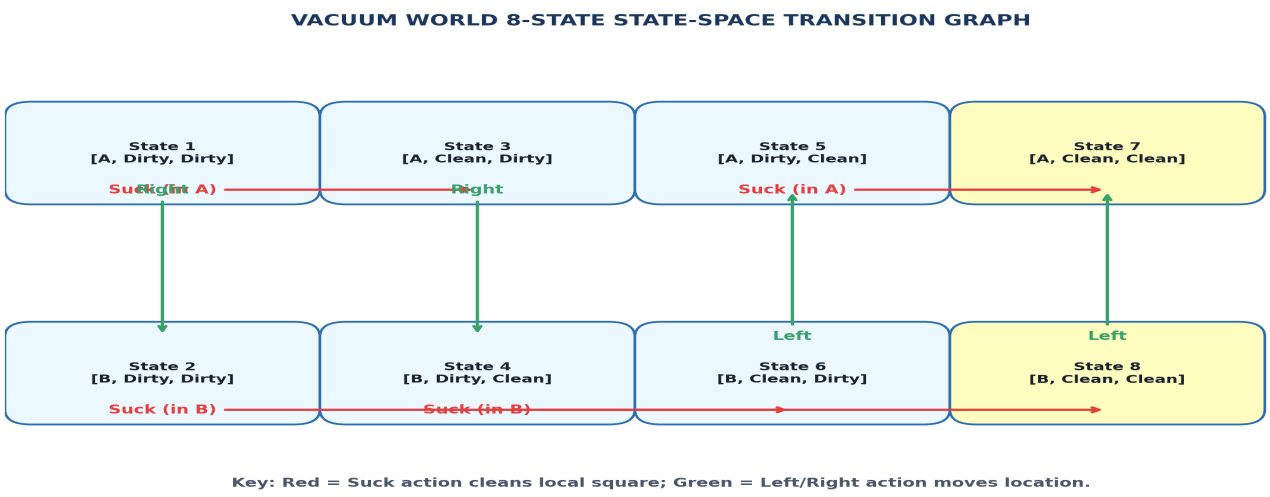


Figure 6.1: Complete Vacuum World 8-State State-Transition Diagram.

Partial Tabulation of Vacuum Agent Function & Reflex Program:

Percept Sequence	Action	Simple Reflex Program Implementation
Percept History / Sequence	Action Selected	Reflex Agent Program Code
[A, Clean]	Right	**function** REFLEX-VACUUM-AGENT([location, status]) **if** status == Dirty **then return** Suck **else if** location == A **then return** Right **else if** location == B **then return** Left

Percept Sequence	Action	Simple Reflex Program Implementation
[A, Dirty]	Suck	
[B, Clean]	Left	
[B, Dirty]	Suck	

7.0 EXAMINATION PREPARATION: SOLVED PYQ BANK & SYLLABUS CHECKLIST

High-Yield Examination Questions & Model Answers:

Q1 [2 Marks]: Define Artificial Intelligence according to John McCarthy.

Model Answer: John McCarthy (1956) defined AI as *'the science and engineering of making intelligent machines, especially intelligent computer programs.'* It focuses on creating computational artifacts capable of acquiring knowledge and reasoning to achieve goals.

Q2 [2 Marks]: State the core architectural equation of an Agent.

Model Answer: The core architectural equation is: **Agent = Architecture + Agent Program**. Here, **Architecture** is the physical computing hardware platform and sensors/actuators, while **Agent Program** is the concrete algorithm executing the agent function $f : P^* \rightarrow A$.

Q3 [2 Marks]: Distinguish between Rationality and Omniscience.

Model Answer: Omniscience knows the *actual outcome* of its actions in advance (impossible in real stochastic worlds). **Rationality** selects actions that maximize *expected performance* based on current evidence and percept sequence to date.

Q4 [5 Marks]: Explain the PEAS framework and construct PEAS for an Automated Taxi Driver.

> **Answer Framework & Marking Scheme:**
> • **PEAS Definition [1 Mark]:** PEAS stands for Performance measure, Environment, Actuators, Sensors.
> • **Performance Measure [1 Mark]:** Safety, destination speed, legal compliance, passenger comfort, max profits.
> • **Environment [1 Mark]:** City streets, highway traffic, pedestrians, weather conditions, police.
> • **Actuators [1 Mark]:** Steering wheel, accelerator, brakes, signal lights, horn, display.
> • **Sensors [1 Mark]:** Cameras, LIDAR, radar, GPS, speedometer, accelerometer, engine sensors.

Q5 [10 Marks]: Explain the architecture of a Learning Agent with a neat diagram. Describe the roles of Critic, Learning Element, Problem Generator, and Performance Element.

> **Model Answer Outline (10 Marks):**
> 1. **Architecture Diagram [3 Marks]:** Draw complete Learning Agent Diagram (Figure 5.5) showing Sensors, Actuators, Critic, Learning Element, Problem Generator, Performance Element, Feedback, Learning Goals, and Environment.
> 2. **Critic [2 Marks]:** Evaluates agent's behavior against an external Performance Standard and generates feedback score.
> 3. **Learning Element [2 Marks]:** Uses feedback from Critic to modify agent program rules and knowledge base.
> 4. **Problem Generator [1.5 Marks]:** Suggests exploratory actions to discover new experiences and better long-term strategies.
> 5. **Performance Element [1.5 Marks]:** The operational agent program that receives percepts and selects external actions.

Unit 1 Topic-by-Topic Syllabus Completeness Checklist:

Syllabus Item	Status	Audit & Diagram Reference
Syllabus Topic Item	Status	Verification & Diagram Cross-Reference
1. Definitions & Four Approaches to AI	COMPLETE	Covered in Section 1.1 - 1.2; Illustrated in Figure 1.1
2. Turing Test & Total Turing Test	COMPLETE	Covered in Section 1.3; Illustrated in Figure 1.2
3. Modeling, Inference, Learning & AI-ML-DL	COMPLETE	Covered in Section 2.1 - 2.2; Illustrated in Figure 2.1
4. Intelligent Agents, P*, f: P* $\rightarrow$ A, Agent Eq.	COMPLETE	Covered in Section 3.1 - 3.2; Illustrated in Figure 3.1
5. Concept of Rationality vs. Omniscience	COMPLETE	Covered in Section 3.4 - 3.5; Illustrated in Figure 3.2
6. PEAS Framework & 6 Real-World Domains	COMPLETE	Covered in Section 4.1 - 4.2; Illustrated in Figure 4.1
7. 7 Task Environment Properties & Matrix	COMPLETE	Covered in Section 4.3 - 4.4; Illustrated in Figure 4.2
8. Simple Reflex Agent Architecture	COMPLETE	Covered in Section 5.1; Illustrated in Figure 5.1
9. Model-Based Reflex Agent Architecture	COMPLETE	Covered in Section 5.2; Illustrated in Figure 5.2
10. Goal-Based Agent Architecture	COMPLETE	Covered in Section 5.3; Illustrated in Figure 5.3
11. Utility-Based Agent Architecture	COMPLETE	Covered in Section 5.4; Illustrated in Figure 5.4
12. Learning Agent Architecture (4 Components)	COMPLETE	Covered in Section 5.5; Illustrated in Figure 5.5
13. Vacuum World 8-State Transition Diagram	COMPLETE	Covered in Section 6.1 - 6.2; Illustrated in Figure 6.1